

IOT ASSIGNMENT

By Vidish Gupta

Q1) You are building a smart irrigation system that must monitor soil moisture, control a water pump, and send data to a mobile application every 5 seconds. You have both an Arduino UNO and an ESP32. Which controller would you choose and why? Would your answer change if internet connectivity was not required?

ANS:

I would definitely pick the ESP32 here. Since the system needs to send data to a mobile app every 5 seconds, the built-in Wi-Fi on the ESP32 makes this incredibly easy. If I used an Arduino UNO, I would have to wire up a separate ESP8266 or Ethernet shield, which just adds unnecessary complexity, wiring mess, and cost. Plus, the ESP32 has more processing power to handle the network stack without freezing the pump control.

If internet connectivity was not required, my answer would change slightly. An Arduino UNO would be perfectly fine for basic local automation (just reading moisture and triggering a relay). However, I would still lean toward the ESP32 because we could use its built-in Bluetooth/BLE to connect directly to a local mobile app without needing the internet.

Q2) A smart factory contains 500 sensors distributed across a 2 km area. Compare Wi-Fi, Bluetooth, LoRa, and MQTT-based communication. Which protocol or combination of protocols would you choose and why?

ANS:

For a massive 2 km area with 500 sensors, Wi-Fi and Bluetooth are completely out of the question. Wi-Fi consumes too much power and its range is way too short, while Bluetooth is even more limited in distance.

I would choose a combination of LoRaWAN and MQTT.

LoRa is perfect for the physical layer because it easily covers the 2 km range, penetrates factory walls, and can handle hundreds of nodes transmitting small amounts of sensor data without draining batteries. The sensor nodes would send data via LoRa to a central gateway in the factory. That single gateway would then be connected to the factory's main network (via Ethernet or Wi-Fi) and use MQTT to publish all that data to the main server. MQTT is lightweight and perfect for organizing data from 500 different topics (sensors).

Q3) A temperature sensor suddenly starts showing impossible values (e.g., -120°C or 300°C). How you would determine whether the problem lies in the sensor, wiring, power supply, ADC, or software.

ANS:

When a sensor throws crazy values like -120°C or 300°C , it is usually a hardware or data interpretation issue, not the actual environment. Here is how I would isolate it:

Software: I would check the datasheet. Many digital sensors (like DHT series) return specific error codes like -999 when they fail to read. If my code is parsing an error code as raw data, that's a software bug.

Wiring: I would physically check the ground and data wires. A loose ground connection often leaves the data pin "floating," causing the microcontroller to read random maximum or minimum values.

Power Supply: I would use a multimeter to measure the voltage exactly at the sensor's pins. If there is a massive voltage drop over a long wire, an analog sensor's output will be completely skewed.

ADC: To test the board's ADC, I would disconnect the sensor and short the ESP's analog pin directly to ground. If it doesn't read 0, the ADC on the microcontroller is fried.

Sensor: If all the above pass, I would swap the sensor with a known working one. If the new one works, the original sensor is dead.

Q4) An ESP32-based weather station must operate on battery power for six months without maintenance. What hardware and software modifications would you make to maximize battery life?

ANS:

Getting six months out of an ESP32 is tough because it's a power-hungry chip, but it's doable with aggressive optimization.

Hardware Modifications: I would physically desolder or break off the power indicator LEDs on the dev board. I'd also bypass the standard inefficient voltage regulator (like the AMS1117) and use an external ultra-low quiescent current regulator (like an HT7333). For the battery, a high-capacity 18650 cell or a LiPo pack is a must.

Software Modifications: Deep sleep is mandatory. I would write the code so the ESP32 wakes up, reads the sensor, connects to Wi-Fi using a static IP (to skip the time-consuming DHCP negotiation), sends the data, and immediately goes back to deep sleep. The Wi-Fi radio should only be on for a couple of seconds a day.

Q5) An automatic braking system in a vehicle requires communication with almost zero delay. Which communication method would you choose and why?

ANS:

I would choose CAN bus (Controller Area Network).

Wireless protocols like Wi-Fi or Bluetooth are out of the question because they are prone to interference, packet loss, and unpredictable

delays. CAN bus is a wired standard designed specifically for automotive environments. It is deterministic, meaning it guarantees that critical messages (like a brake command) get priority and arrive within a strict, predictable timeframe. It is also highly resilient to electrical noise generated by the vehicle's engine.

Q6) You need to measure the average temperature of a classroom using only one sensor. Where would you place the sensor and why? Explain how poor placement could affect the accuracy of your measurements. Assume the classroom to be a square-base cuboid of side length “a” and height “b”.

ANS:

The best placement is on an inner wall, right in the middle of the room horizontally, and at roughly head height (about 1.5 meters from the floor).

Why: Heat rises, so if you place it near the ceiling (height b), it will read much hotter than what the students actually feel. If placed on the floor, it will read too cold. An inner wall avoids the direct temperature transfer that happens on exterior walls.

Poor Placement Effects: Putting the sensor right next to a window will cause false spikes due to direct sunlight. Placing it right under an AC vent will trigger the system to shut off too early because it thinks the whole room is freezing. Placing it near the door will cause massive fluctuations every time someone walks in.

Q7) Under what situations could an analog sensor perform better than a digital one?

ANS:

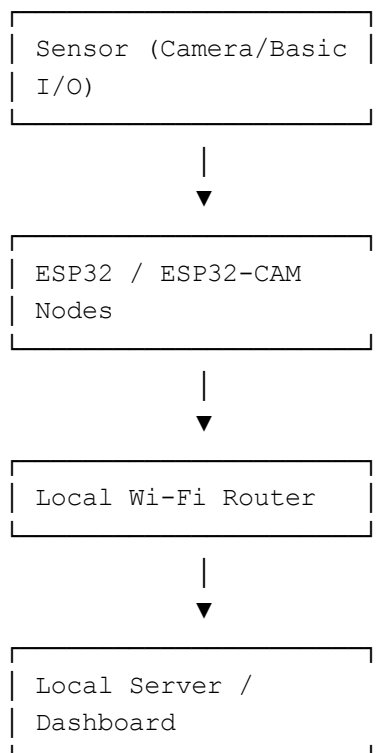
An analog sensor is better in situations requiring absolute zero-latency, continuous monitoring. Digital sensors require communication protocols (like I2C or SPI) which take processing time and have sampling limits.

Analog sensors just output a continuous voltage or current. Also, in harsh industrial environments with lots of electrical noise, a robust analog 4-20mA current loop can transmit a signal over very long cables much more reliably than standard digital signals, which might get corrupted over distance.

Q8) Draw the complete architecture/block diagram of your IoT project, beginning from the sensor and ending at the user interface. Identify every possible point where a failure could occur and suggest one solution for each.

ANS:

Architecture Flow:



Point of Failure 1: ESP32-CAM brownouts. When the camera and Wi-Fi turn on simultaneously, it draws a massive current spike, often crashing the board.

Solution: Add a high-capacity capacitor (like 470uF) directly across the 5V and GND pins of the module to handle power spikes.

Point of Failure 2: Wi-Fi connection drops. The router might restart or temporarily lose signal, leaving the prototype completely disconnected.

Solution: Implement a non-blocking auto-reconnect logic in the loop() function to scan and reconnect automatically without freezing the rest of the code.

Point of Failure 3: Hardware freezing due to static or random errors.

Solution: Enable the hardware Watchdog Timer (WDT) to automatically reset the chips if the code gets stuck in an infinite loop.

Q9) If an attacker gains access to your IoT device through the Wi-Fi network, what kinds of attacks could they perform? Suggest at least five methods to improve the security of the system (with applicable solutions, do not cite far-fetched ideas)

ANS:

If an attacker gains access to an IoT device through the Wi-Fi network, they can perform several malicious activities that may compromise the device, data, and the entire network.

Possible Attacks

1. Unauthorized Device Control

- a. The attacker can turn devices ON/OFF or change their settings (e.g., control smart lights, locks, or pumps).

2. Data Theft

- a. They can steal sensor data, user information, passwords, or API keys stored on the device.

3. Malware Installation

- a. The attacker can install malicious software to damage the device or make it part of a botnet.

4. Denial of Service (DoS)

- a. They can flood the device with requests, causing it to become slow or stop responding.

5. Network Attack

- a. Once inside the IoT device, the attacker may try to access other devices connected to the same Wi-Fi network.

6. Firmware Modification

- a. They may replace the original firmware with malicious firmware, allowing permanent control of the device.

To improve the security of the system, the following measures should be implemented:

1. Use WPA3/WPA2 Wi-Fi security with a strong, unique password.
2. Replace default credentials with strong passwords and enable multi-factor authentication where possible.
3. Encrypt communication using HTTPS/TLS or MQTTS.
4. Regularly update the device firmware to fix security vulnerabilities.
5. Place IoT devices on a separate or guest network to isolate them from other devices.
6. Disable unused ports and services to reduce the attack surface.
7. Implement proper user authentication and access control so only authorized users can access the device.

These practical security measures significantly reduce the risk of unauthorized access and improve the overall security of the IoT system.

Q10) Your ESP32 must simultaneously interface with a DHT11 sensor, an ultrasonic sensor, an OLED display, a relay module, and a soil moisture sensor. What hardware or software conflicts could occur, and how would you solve them?

ANS:

When interfacing an ESP32 with a DHT11 sensor, ultrasonic sensor, OLED display, relay module, and soil moisture sensor, several hardware and software conflicts may occur. Hardware conflicts include GPIO pin

conflicts, insufficient power for the relay, voltage level mismatches between 3.3 V and 5 V devices, and electrical noise from the relay affecting sensor readings. These can be solved by assigning unique GPIO pins, using an external power supply for high-current devices, using logic level converters where required, and employing opto-isolated relay modules with proper grounding and decoupling. Software conflicts include timing issues with the DHT11 and ultrasonic sensor and blocking delays that prevent simultaneous operation. These can be solved by reading sensors sequentially, using `millis()` or FreeRTOS tasks instead of `delay()`, and updating the OLED and relay only when necessary. These measures ensure reliable and conflict-free operation of all peripherals.

Q11) An ESP32 uploads sensor data to the cloud every minute. The internet connection fails for 12 hours. Design a strategy that ensures no important data is permanently lost.

ANS:

To ensure that no sensor data is permanently lost during a 12-hour internet outage, the ESP32 should store the data locally and upload it once the connection is restored.

Strategy

1. Store Data Locally

- Save each sensor reading (timestamp + sensor values) in the ESP32's flash memory (SPIFFS/LittleFS) or on a microSD card.
- Since data is collected every minute, a 12-hour outage results in only 720 readings, which can easily be stored.

2. Check Internet Connection

- Before sending data, check if Wi-Fi and the cloud server are reachable.
- If connected, upload the latest data.
- If not connected, continue storing readings locally.

3. Maintain a Queue

- Keep unsent data in chronological order (FIFO queue).
- Mark each record as "pending."

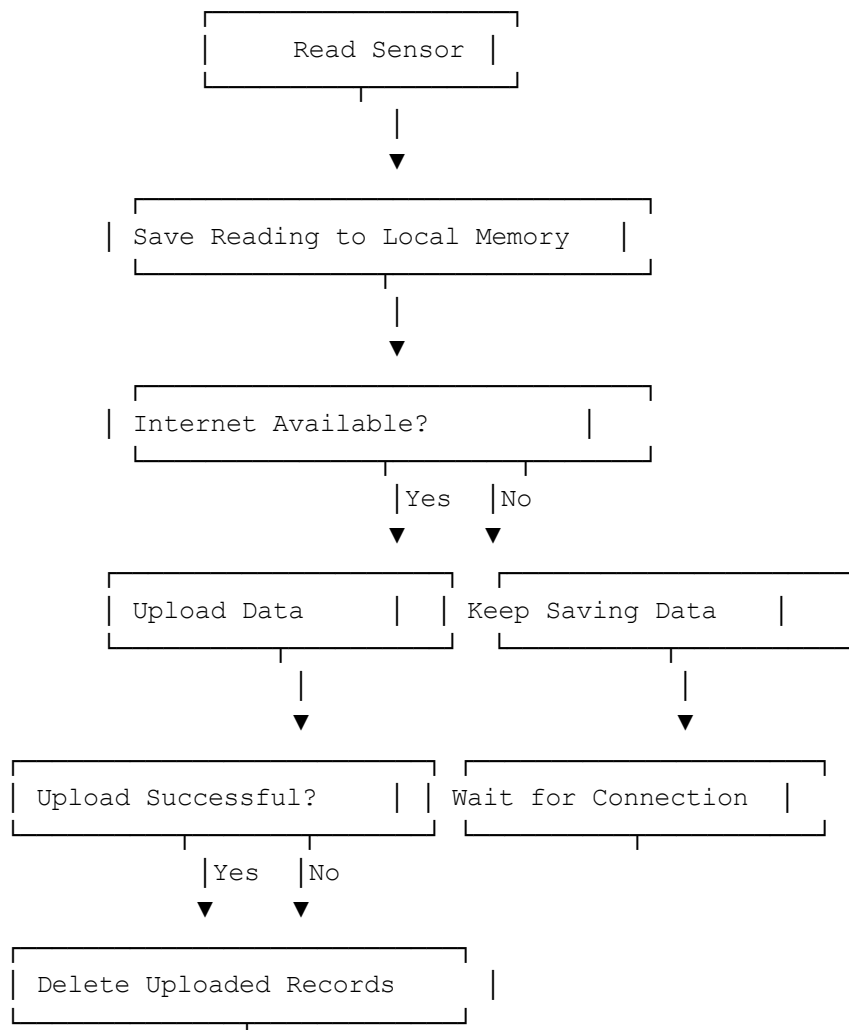
4. Upload Missing Data

- When the internet is restored, send the oldest pending records first.
- After the cloud confirms successful receipt, delete those records from local storage.

5. Prevent Data Loss

- Save data immediately after taking each reading.
- This ensures that even if the ESP32 resets during the outage, previously stored data remains available.

WORKFLOW:



Q12) Your IoT device must operate continuously for five years without human intervention. What changes would you make to both the hardware and software to improve long-term reliability?

ANS:

To ensure an IoT device operates continuously for five years without human intervention, both the hardware and software must be designed for high reliability, low maintenance, and fault tolerance.

Hardware Improvements

1. Use High-Quality Components

- Select industrial-grade sensors and components with long operating life.

2. Reliable Power Supply

- Use a stable regulated power supply with surge, over-voltage, and reverse-polarity protection.
- Include a backup battery or solar power if required.

3. Environmental Protection

- Enclose the device in a waterproof and dustproof (e.g., IP65/IP67) enclosure.
- Protect against heat, moisture, and corrosion.

4. Watchdog Hardware

- Enable the ESP32 watchdog timer so the device automatically resets if it becomes unresponsive.

5. Reduce Hardware Stress

- Use relay modules and components within their rated limits to prevent overheating and premature failure.

Software Improvements

1. Watchdog Timer

- Restart the system automatically if the program hangs.

2. Error Handling

- Detect sensor failures, communication errors, and invalid readings, and recover automatically.

3. Automatic Reconnection

- If Wi-Fi or cloud connectivity is lost, keep retrying until the connection is restored.

4. Data Backup

- Store important data locally during network failures and upload it when the connection returns.

5. Over-the-Air (OTA) Updates

- Support remote firmware updates to fix bugs and add improvements without physical access.

6. Efficient Memory Management

- Prevent memory leaks by freeing unused memory and avoiding unnecessary dynamic allocation.

7. Health Monitoring

- Periodically monitor battery voltage, sensor status, memory usage, and system uptime.
- Restart or report faults if abnormal conditions are detected.

Q13) Can an IoT system be considered "smart" if it does not use Artificial Intelligence? Justify your answer with suitable examples. What differentiates “smart” from “intelligent”?

ANS:

Yes, an IoT system can be considered "smart" even without using Artificial Intelligence (AI). Smartness comes from the ability to **sense**, communicate, and automatically respond based on predefined rules. AI is only required when the system needs to learn, predict, or make complex decisions.

Smart vs Intelligent

Smart System	Intelligent System
Works using predefined rules or conditions. Does not require AI.	Learns from data and improves its decisions over time. Uses AI or Machine Learning algorithms.
Performs automatic actions based on programmed logic.	Makes predictions, recognizes patterns, and adapts to new situations.
Less complex and easier to implement.	More complex and computationally intensive.

Example of a Smart IoT System (Without AI)

Smart Irrigation System

- A soil moisture sensor measures the moisture level.
- If the moisture falls below a threshold, the ESP32 automatically turns ON the water pump.
- When the moisture reaches the required level, the pump turns OFF.

This system is smart because it senses, decides, and acts automatically, but it does not use AI.

Example of an Intelligent IoT System (With AI)

AI-Based Smart Irrigation

- The system collects soil moisture, weather forecasts, temperature, and rainfall history.

- An AI model predicts future water requirements.
- It decides the optimal watering schedule to save water while maintaining crop health.

This system is intelligent because it learns from data and makes predictions rather than following fixed rules.

Q14) Compare polling and interrupt-based programming in embedded systems. In which situations would interrupts significantly improve the performance of an IoT device?

ANS:

Polling	Interrupt-Based Programming
The CPU continuously checks whether an event has occurred.	The CPU performs other tasks and is interrupted only when an event occurs.
Wastes CPU time because devices are checked repeatedly.	Efficient use of CPU time since it responds only when needed.
Higher power consumption.	Lower power consumption.
Slower response if the polling interval is long.	Very fast response to important events.
Easier to implement.	Slightly more complex to implement.

Best for non-critical or periodic tasks.

Best for real-time and time-critical tasks.

Situations Where Interrupts Improve IoT Performance

1. Motion Detection (PIR Sensor)

- a. The ESP32 remains in sleep mode.
- b. A PIR sensor interrupt wakes the ESP32 immediately when motion is detected.

2. Emergency Button or Alarm

- a. A button press generates an interrupt, allowing an instant response without continuously checking the button.

3. Fire or Smoke Detection

- a. As soon as the sensor detects danger, an interrupt immediately activates the alarm and sends a notification.

4. Ultrasonic Sensor Echo Detection

- a. Interrupts accurately measure the echo pulse duration for precise distance measurement.

5. Serial Communication (UART/I²C/SPI)

- a. Interrupts notify the processor when new data arrives, preventing data loss while the CPU performs other tasks.

6. Low-Power Battery Devices

- a. The microcontroller stays in deep sleep and wakes only when an interrupt occurs, greatly extending battery life.

Q15) Every engineering design involves trade-offs. Looking at your own IoT project, identify three design decisions where you sacrificed one aspect (such as cost, speed, power, accuracy, or complexity) to improve another. Explain whether you believe those trade-offs were justified.

ANS:

My prototype required making specific compromises:

Sacrificing Range for Bandwidth: I chose to use Wi-Fi instead of a long-range protocol like LoRa. While LoRa would give me massive range, I needed the high bandwidth of Wi-Fi to transmit the image data from the ESP32-CAM.

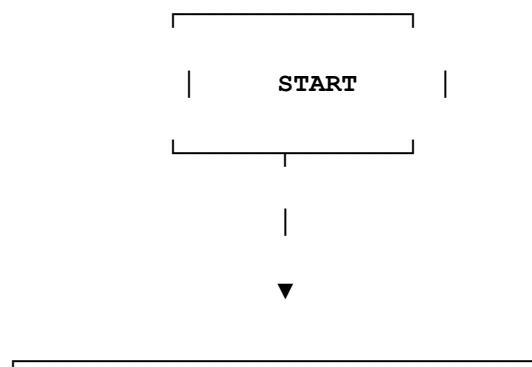
Sacrificing Location Tracking for Simplicity & Power: I decided not to include a GPS module in this build. Since this is just a prototype phase, omitting the GPS significantly reduced the power draw and hardware complexity, allowing me to focus strictly on the core networking logic.

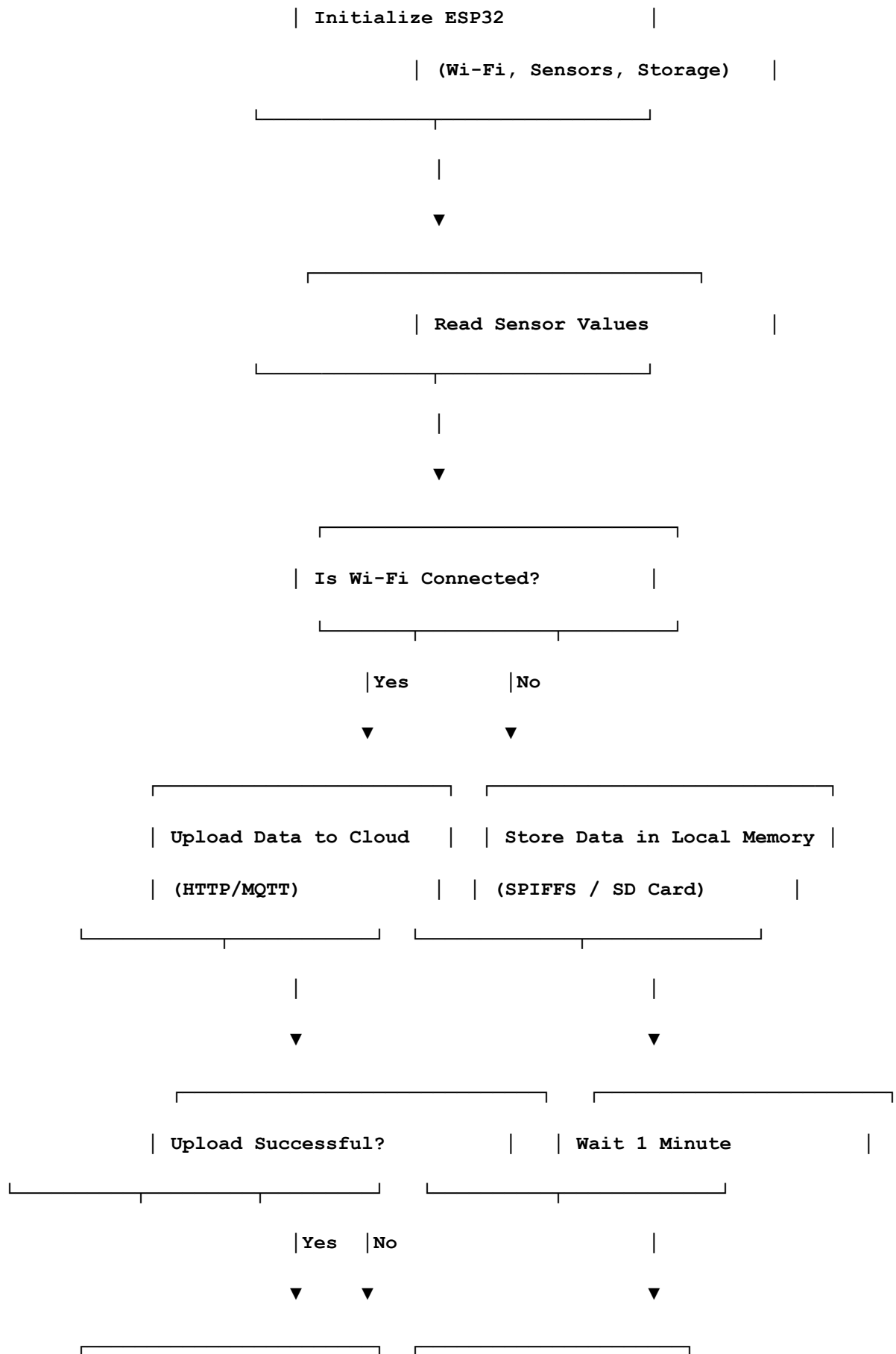
Sacrificing Board Space for Modularity: Instead of trying to find one single, expensive board that does everything, I split the project between a standard ESP32 and a separate ESP32-CAM. This makes the physical footprint larger and slightly less power-efficient.

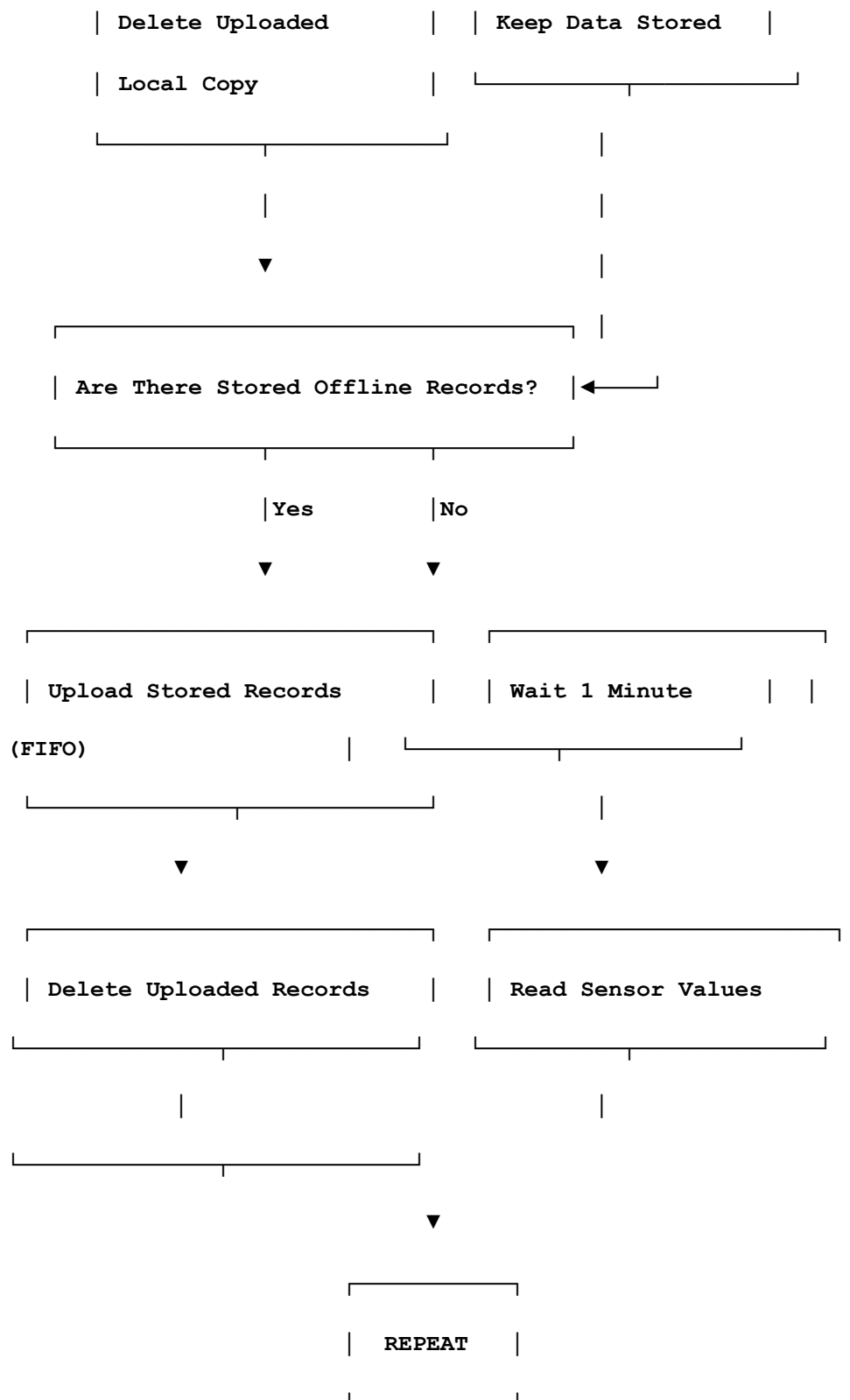
The trade-off is justified because it makes debugging and writing the firmware much easier during the prototype phase.

Q16) Create a flowchart for an IoT system that stores sensor readings locally whenever Wi-Fi is unavailable and automatically uploads the stored data once the connection is restored. Mention all the programs/protocols/I-O used in the system.

ANS:







The IoT system continuously reads sensor data at fixed intervals. If the Wi-Fi connection is available, the ESP32 uploads the current reading to the cloud using HTTP or MQTT. If Wi-Fi is unavailable, the sensor data is stored locally in SPIFFS, LittleFS, or a microSD card along with a

timestamp. When the internet connection is restored, the ESP32 automatically uploads all stored readings in chronological order and deletes them from local storage only after receiving successful acknowledgment from the cloud.

The system uses Arduino IDE for programming, GPIO for sensor interfacing, Wi-Fi for communication, HTTP/MQTT as the communication protocol, SPIFFS/LittleFS/microSD for local storage, and a cloud database for permanent data storage. This ensures that no sensor readings are lost during network failures.

Q17) Draw a flowchart showing how an ESP32 performs an OTA (Over-the-Air) firmware update, including error handling if the update fails.

ANS:

